El TAD Pila por dentro

Una lista por la que solo se entra por un extremo

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Construir el TAD Pila sobre el TAD Lista, escogiendo el extremo que sirve de tope (OA2, OA3).
- Explicar por qué el mismo diseño cuesta $\Theta(1)$ o $\Theta(n)$ según la implementación de lista que haya debajo (OA1, OA3).
- Escribir la pila directamente sobre nodos y decir qué gana y qué pierde frente a la versión por composición (OA2).
- Dar el costo en tiempo y en espacio de `apilar`, `desapilar` y `tope` en cada combinación (OA1, OA5).

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

El contrato y su implementación provisional I

La pila atiende por un solo extremo: lo último que entra es lo primero que sale.

```
class Pila {
private:
  Elemento datos[CAPACIDAD];
  int n;
public:
  void apilar(Elemento e) {
    assert(n < CAPACIDAD);
    datos[n] = e;
    n = n + 1;
  }
```

El contrato y su implementación provisional II

El tope es `datos[n - 1]`, así que apilar y desapilar escriben una casilla y mueven el contador: $\Theta(1)$ las dos.

Lo que esa implementación arrastra

`CAPACIDAD` se fija al compilar. El elemento que no cabe detiene el programa, y las casillas que sobran quedan reservadas aunque la pila esté vacía.

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

No hay que volver a manejar punteros I

La lista enlazada ya sabe insertar, eliminar y consultar por posición. Una pila no necesita más que eso en un solo extremo, así que puede construirse *usando* una lista en vez de repetir el manejo de nodos.

```
class Pila {
private:
  Lista l;
};
```

Lo único que hay que decidir

Cuál extremo de la lista es el tope. Si es la posición 0, apilar es `insertar(0, e)` y desapilar es `eliminar(0)`. Si es el final, apilar es `agregar(e)` y desapilar es `eliminar(tamano() - 1)`.

No hay que volver a manejar punteros II

Pregunta

Las dos decisiones dan la misma pila y cumplen el mismo contrato. ¿Da lo mismo escoger cualquiera?

No da lo mismo, y depende de lo que haya debajo I

Apilando y desapilando n veces, contando los elementos que se mueven en el arreglo y los nodos que se visitan en la enlazada:

n	arreglo, tope al frente	arreglo, tope al final	enlazada, tope al frente	enlazada, tope al final
100	9 900	0	0	9 702
1000	999 000	0	0	997 002
10000	99 990 000	0	0	99 970 002

Las dos estructuras son el espejo una de la otra

El arreglo es gratis por el final, porque la casilla siguiente ya está, y cuadrático por el frente, porque hay que correr todo. La enlazada es gratis por el frente, porque son dos punteros, y cuadrática por el final, porque hay que caminar hasta allá.

No da lo mismo, y depende de lo que haya debajo II

Lo que esto significa

El costo de `apilar` no es una propiedad del TAD Pila. Es una propiedad de la lista que se escoja debajo y del extremo que se nombre tope. Escribir $O(1)$ al lado de `apilar` sin decir cuál de las dos cosas es afirmar algo que puede ser falso.

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

Tope en la posición 0 I

Con la lista enlazada debajo, el tope va al frente: insertar y eliminar en la posición 0 son dos escrituras de puntero.

```
class Pila {
private:
  Lista l;

public:
  void apilar(Elemento e) {
    l.insertar(0, e);
  }
  // exige !vacia()
  void desapilar() {
    assert(!vacia());
    l.eliminar(0);
  }
```

Tope en la posición 0 II

```
  // exige !vacia()
  Elemento tope() {
    assert(!vacia());
    return l.obtener(0);
  }
  int tamano() {
    return l.tamano();
  }
  bool vacia() {
    return l.vacia();
  }
};
```

Tope en la posición 0 III

Toda la pila son seis llamadas. No aparece un solo `new`, un solo `delete` ni un solo puntero: eso ya lo resolvió la lista, y el destructor de la lista libera los nodos cuando la pila sale de alcance.

Salida

Apilando 5, 8 y 2, desapilando una vez y apilando 9: `2 3 / 8 2 / 9 8 5 / 0 1`

La precondición vive en la pila, no en la lista I

`l.eliminar(0)` sobre una lista vacía viola la precondición de la lista, y el `assert` que salta sería el de ella. El contrato de la pila dice que desapilar exige `!vacia()`, así que la comprobación va aquí, donde el programador que usa la pila puede entenderla.

Dos contratos, uno encima del otro

La pila promete menos que la lista: no deja entrar por el medio ni consultar la posición 3. A cambio, garantiza que lo último que entró es lo que sale. Construir un TAD sobre otro es escoger qué parte del de abajo se deja ver.

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

La pila directa sobre nodos I

Si la pila solo toca un extremo, la lista de abajo nunca usa `nodoEn`, ni el contador de posiciones, ni los casos de insertar en el medio. Se puede escribir la cadena directamente.

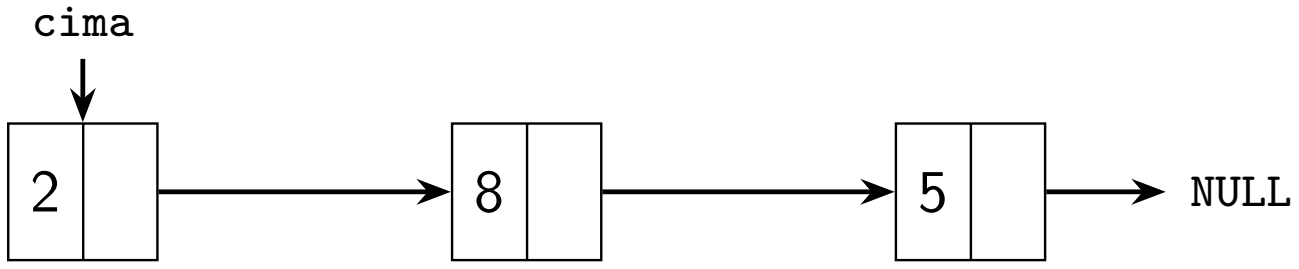
```
class Pila {
private:
  Nodo *cima;
  int n;
public:
  Pila() {
    cima = NULL;
    n = 0;
  }
```

La pila directa sobre nodos II

```
void apilar(Elemento e) {
  Nodo *nuevo = new Nodo;
  nuevo->dato = e;
  nuevo->siguiente = cima;
  cima = nuevo;
  n = n + 1;
}
```

```
// exige !vacia()
void desapilar() {
  assert(!vacia());
  Nodo *muerto = cima;
  cima = cima->siguiente;
  delete muerto;
  n = n - 1;
}
```

La pila directa sobre nodos III

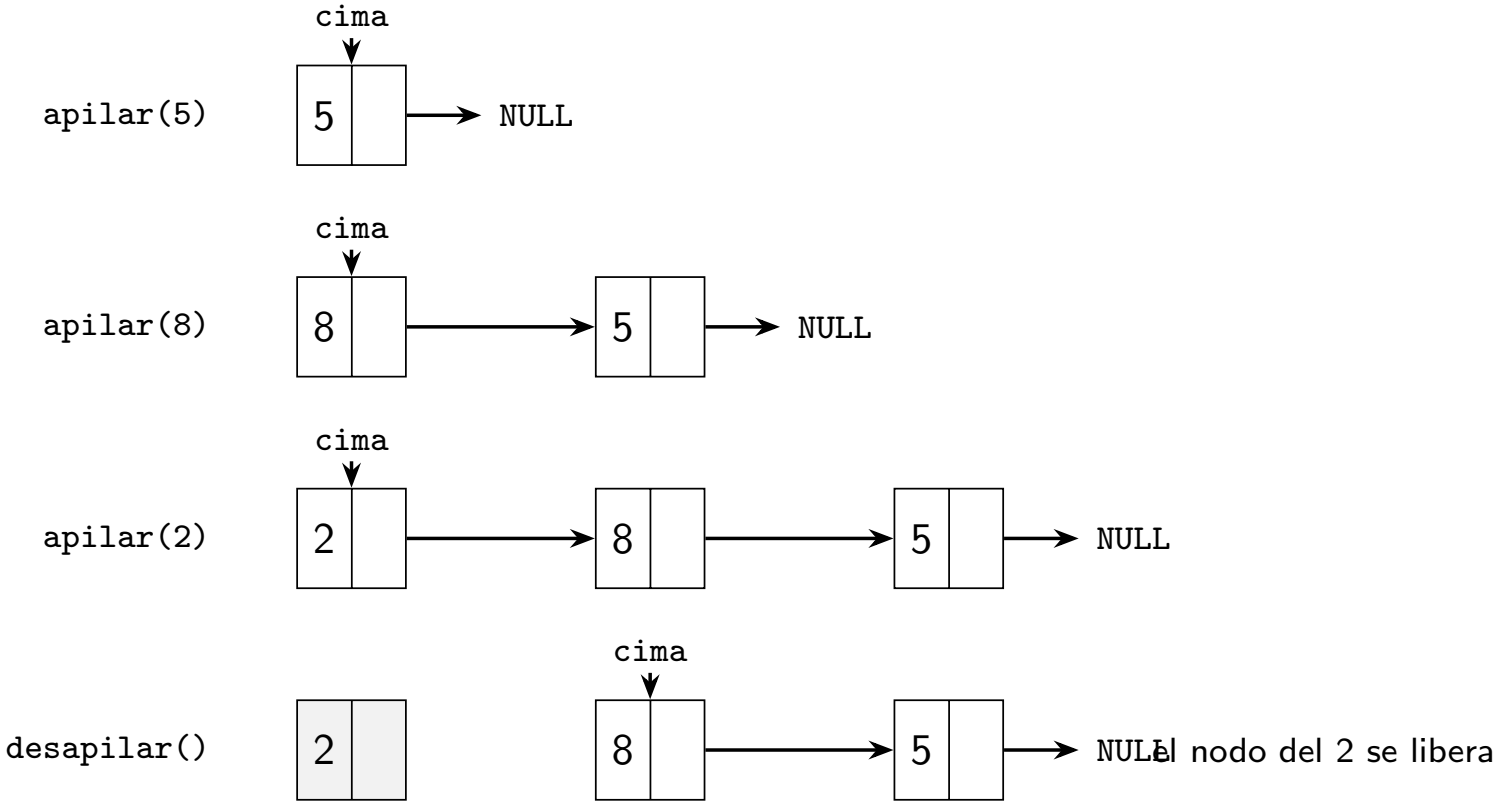


lo último que entró está de primero

No hay `CAPACIDAD` y no hay desbordamiento: la pila crece mientras haya memoria. El `assert` que queda es el de la precondición, no el del techo.

La traza I

Partiendo de la pila vacía, `apilar(5)`, `apilar(8)`, `apilar(2)` y `desapilar()`.



La traza II

Cada paso escribe dos punteros y mueve el contador. El nodo que sale se guarda antes de mover `cima`, porque después de `delete` ya no se puede leer.

El techo que desaparece I

Apilando hasta donde cada una aguanta, con una capacidad de 1000:

intentos	caben en el arreglo	caben en la enlazada	memoria de los nodos
500	500	500	7 KiB
1000	1000	1000	15 KiB
100000	1000	100000	1562 KiB

Lo que cuesta no tener techo

Un `Nodo` ocupa 16 bytes: 4 del entero, 8 del puntero y 4 de relleno para que el puntero quede alineado. El dato es la cuarta parte de lo que se guarda. El arreglo no paga nada de eso, y a cambio reserva sus 1000 casillas aunque la pila esté vacía y se detiene en la 1001.

La escogencia no es cuál es mejor, sino si se sabe de antemano cuántos elementos van a caber.

Cuál de las dos escribir I

Las dos dan exactamente la misma salida sobre el mismo programa. La diferencia está en otra parte.

Sobre la lista	Directa sobre nodos
seis llamadas, ningún puntero a la vista	unas treinta líneas con `new` y `delete`
el destructor de la lista libera	hay que escribir el destructor
un salto de llamada por operación	toca la cadena de una vez
si cambia la lista, cambia el costo	el costo está a la vista

La primera reutiliza trabajo ya hecho y probado, que es lo que se busca al tener un TAD. La segunda se escribe cuando la estructura

Cuál de las dos escribir II

de abajo tiene operaciones que esta no va a usar nunca, y cargarlas sale más caro que escribir las cinco que sí.

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

Las cuatro formas de tener una pila I

Operación	Arreglo con tope	Lista sobre arreglo, tope al frente	Lista enlazada, tope al frente	Directa sobre nodos
`apilar(e)`	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
`desapilar()`	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
`tope()`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
`tamano()`, `vacia()`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
espacio de la estructura	$\Theta($`CAPACIDAD`$)$	$\Theta($`CAPACIDAD`$)$	$\Theta(n)$	$\Theta(n)$
espacio por elemento	el dato	el dato	el dato y un puntero	el dato y un puntero
techo de elementos	`CAPACIDAD`	`CAPACIDAD`	la memoria	la memoria

Por qué

El arreglo con tope es $\Theta(1)$ porque escribe la casilla `n` y la suelta. La lista sobre arreglo con el tope al frente corre todo en cada operación. La enlazada por el frente escribe dos punteros. La directa hace lo mismo que la enlazada sin el salto de llamada.

La pila es el TAD donde la lista enlazada no pierde nada, porque nunca pregunta por una posición del medio: lo único que paga es un puntero por elemento.

Para el juez: UVa 732 — Anagrams by Stack I

UVa 732 — Anagrams by Stack,
`https://vjudge.net/problem/UVA-732`
Envío: `https://vjudge.net/problem/UVA-732`

Qué pide

Dos palabras por caso, la de entrada y la de salida. Hay que imprimir todas las secuencias de `i` (meter la siguiente letra a la pila) y `o` (sacar la del tope y escribirla) que convierten la primera en la segunda, en orden alfabético y entre corchetes.

Entrada, salida y límites

Las palabras no pasan de unos pocos caracteres. Cada línea de salida lleva las letras `i` y `o` separadas por espacio.

Para el juez: UVa 732 — Anagrams by Stack II

Cómo atacarlo con lo de hoy

En cada paso hay a lo sumo dos jugadas: meter la siguiente letra, o sacar el tope si coincide con la letra que toca escribir. Se prueban las dos y se deshace la que no lleve a nada, lo que exige que apilar y desapilar sean baratos: se hacen y se deshacen miles de veces. La pila enlazada con el tope al frente da $\Theta(1)$ en las dos.

Las trampas

- El orden alfabético sale solo si en cada paso se intenta primero `i` y después `o`, porque la `i` va antes que la `o`.
- Al deshacer una jugada hay que devolver la pila al estado exacto en que estaba, incluida la letra que se había sacado.
- Una palabra de salida que no sea anagrama de la de entrada no imprime ninguna línea, pero sí imprime los corchetes.

El programa: lo que las llamadas comparten I

Las dos palabras, su largo y la secuencia en construcción no cambian durante la búsqueda, así que viven afuera de `buscar`.

```
#include <cstdio>
#include <cstring>
#include "pila_nodos.h"

const int MAXIMO = 100;  // letras que caben en cada palabra

// + 1 para el '\0' que cierra cada cadena
char entrada[MAXIMO + 1];
char salida[MAXIMO + 1];
// las dos palabras miden lo mismo cuando se busca
int largo;
// la secuencia en construccion: 2 * largo jugadas
char jugadas[2 * MAXIMO];
```

El programa: lo que las llamadas comparten II

```
void imprimir() {
  int k = 0;
  while (k < 2 * largo) {
    if (k > 0) {
      printf(" ");
    }
    printf("%c", jugadas[k]);
    k = k + 1;
  }
  printf("\n");
}
```

`jugadas` usa sus primeras `2 * largo` casillas y se escribe por posición: cada llamada pisa la casilla de su propia jugada y las anteriores quedan como las dejaron las llamadas que venían antes.

Las dos jugadas, y cómo se deshacen I

```
// metidas: letras de la entrada que ya estan en la pila o salieron de
↪  ella.
// escritas: letras de la salida ya escritas.
// La jugada que se decide aqui es la numero metidas + escritas.
void buscar(Pila &p, int metidas, int escritas) {
  if (escritas == largo) {
    imprimir();
  } else {
    if (metidas < largo) {
      p.apilar(entrada[metidas]);
      jugadas[metidas + escritas] = 'i';
      buscar(p, metidas + 1, escritas);
      p.desapilar();
    }
    if (!p.vacia() && p.tope() == salida[escritas]) {
      Elemento letra = p.tope();
      p.desapilar();
      jugadas[metidas + escritas] = 'o';
      buscar(p, metidas, escritas + 1);
      p.apilar(letra);
    }
  }
}
```

Las dos jugadas, y cómo se deshacen II

La pila entra por referencia: hay una sola en toda la búsqueda y es la que se arma y se desarma. Las dos condiciones son las jugadas legales: se mete mientras queden letras de la entrada, y se saca solo si el tope es la letra que toca escribir.

Un caso por línea, y los corchetes siempre I

```
int main() {
  while (scanf("%s %s", entrada, salida) == 2) {
    printf("[\n");
    if (strlen(entrada) == strlen(salida)) {
      largo = strlen(entrada);
      Pila p;
      buscar(p, 0, 0);
    }
    printf("]\n");
  }
  return 0;
}
```

Un caso por línea, y los corchetes siempre II

Cada vuelta lee las dos palabras con `scanf` en los arreglos de `char`, y `strlen` da el largo de cada una. El ciclo termina cuando `scanf` ya no encuentra dos palabras. Los corchetes se escriben antes y después de la búsqueda, con secuencias o sin ellas. Cuando las palabras no miden lo mismo no se busca: ninguna secuencia de jugadas puede escribir más letras de las que entraron.

Lo que sale con el ejemplo del enunciado

`madam` a `adamm` da 4 secuencias y `bahama` a `bahama` otras 4. `long` a `short` no da ninguna y aun así imprime los corchetes. `eric` a `rice` da una sola.

La `i` primero, y el estado que se devuelve I

De dónde sale el orden alfabético

Dos secuencias del mismo caso tienen las mismas $2n$ jugadas y se comparan letra por letra: decide la primera posición en que difieren, y ahí la `i` va antes que la `o`. En el árbol de la búsqueda, dos secuencias que comparten las primeras k jugadas se separan en el estado que decide la jugada k, y de ese estado la rama de la `i` se recorre completa antes de abrir la de la `o`. Todo lo que se imprime en la primera rama es entonces menor que todo lo de la segunda, y no queda nada por ordenar al final.

La `i` primero, y el estado que se devuelve II

`foo a oof`

Salen dos secuencias, en este orden:

1. `i i i o o o`
2. `i i o i o o`

Coinciden en las dos primeras jugadas; en la tercera, la de arriba mete y la de abajo saca.

La `i` primero, y el estado que se devuelve III

Qué es deshacer una jugada

Devolver la pila al estado exacto que tenía antes: el mismo contenido, en el mismo orden y con el mismo tope. Al volver de la rama de la `i`, `p.desapilar()` saca la letra que esa rama había metido. Al volver de la rama de la `o`, `p.apilar(letra)` mete de nuevo la que se había sacado.

Por eso la letra se guarda antes de sacarla

`desapilar` no devuelve el elemento, y la letra se va con el nodo que se borra. La línea `Elemento letra = p.tope();` es lo que permite reponerla. Sin ella, la rama de la `o` termina con un elemento de menos y la siguiente explora un estado que no corresponde a ninguna secuencia de jugadas.

Lo que cuesta encontrarlas todas I

Con n el largo de cada palabra.

Qué	Costo	Por qué
meter o sacar una letra	$\Theta(1)$	`apilar` y `desapilar` escriben la cabeza de la cadena
deshacer esa jugada	$\Theta(1)$	es la operación contraria, una sola vez
mirar si se puede sacar	$\Theta(1)$	`vacia()` lee el contador y `tope()` la cabeza
imprimir una secuencia	$\Theta(n)$	se recorren las $2n$ jugadas
estados explorados	$O(4^n)$	dos ramas por estado, $2n$ jugadas de profundidad
tiempo total	$O(n \cdot 4^n)$	los estados, más $\Theta(n)$ por cada secuencia impresa
espacio de la recursión	$\Theta(n)$	a lo sumo $2n + 1$ llamadas anidadas
espacio de la pila	$\Theta(n)$	nunca hay más de n letras adentro
espacio de `jugadas`	$\Theta(n)$	$2n$ casillas, reservadas una vez por caso

Lo que cuesta encontrarlas todas II

De dónde sale el 4^n

Cada estado decide la jugada número `metidas + escritas`, que va de 0 a $2n$, y abre a lo sumo dos ramas. Un árbol binario de profundidad $2n$ tiene $O(2^{2n}) = O(4^n)$ nodos, y esa es la cota de los estados que se visitan.

Cuando todas las letras son iguales

Ninguna jugada o queda descartada por la prueba del tope, así que se imprime toda secuencia legal: el número de Catalan $C_n = \frac{1}{n+1}\binom{2n}{n}$. Con $n = 4$ son 14, con $n = 5$ son 42 y con $n = 10$ son 16 796.

Lo que ahorra la prueba del tope

Con letras distintas la poda corta casi todo: de las 42 secuencias legales de cinco letras, `madam` a `adamm` deja 4.

Contenido I

1 La pila que ya tienen

2 Una pila es una lista por un solo extremo

3 La pila sobre la lista enlazada

4 Sin la lista de por medio

5 Lo que cobra cada operación

6 Ejercicios

Ejercicio en parejas: el tope al final I

Escriba la pila sobre la lista enlazada poniendo el tope en el *final*: `apilar` es `agregar(e)` y `desapilar` es `eliminar(tamano() - 1)`.

Las preguntas

1. ¿Cuánto cuesta cada una sobre la lista enlazada simple?
2. Con el puntero al último que se le agregó a la lista, ¿cuál de las dos baja de costo?
3. ¿Y la otra? Diga por qué.
4. ¿Con cuál variante de la lista quedarían las dos en $\Theta(1)$?

Ejercicio en parejas: el tope al final II

Lo que debe salir

Sobre la simple, las dos son $\Theta(n)$: hay que caminar hasta el final. Con el puntero al último, `agregar` baja a $\Theta(1)$, pero `eliminar` del final **no**: para desenlazar el último hay que reescribir el campo `siguiente` del penúltimo, y a ese no se llega sin recorrer. Guardar el final sirve para poner, no para quitar.
Las dos quedan en $\Theta(1)$ con la doblemente enlazada, donde el último sabe quién es su anterior.

Y sin embargo

Nada de esto hacía falta: con el tope al frente, la lista simple ya daba $\Theta(1)$ en las tres operaciones. Escoger bien el extremo ahorró toda la estructura adicional.

Ejercicio en parejas: copiar una pila I

Escriba una función que reciba una pila y devuelva otra con el mismo contenido en el mismo orden, **dejando la original como estaba**. Solo se pueden usar las operaciones del contrato: `apilar`, `desapilar`, `tope`, `tamano` y `vacia`.

Las preguntas

1. ¿Cuántas pilas auxiliares hacen falta?
2. Si usa una sola, ¿en qué orden queda la copia?
3. ¿Cuántas operaciones hace en total con n elementos?
4. ¿Qué operación del TAD Lista haría esto trivial, y por qué la pila no la ofrece?

Ejercicio en parejas: copiar una pila II

Lo que debe salir

Vaciar la original en una auxiliar la deja al revés, y vaciar la auxiliar en dos pilas a la vez reconstruye la original y deja la copia en el orden correcto. Son dos recorridos: $\Theta(n)$ operaciones y $\Theta(n)$ de espacio para la auxiliar.

Con `obtener(i)` de la lista se recorrería sin desarmar nada. La pila no la ofrece a propósito: el contrato existe para impedir que alguien lea por el medio, y el precio de esa garantía es que copiar cuesta desarmar.

Propuestos para la casa I

1. **La pila con techo.** Agregue un límite máximo de elementos a la pila enlazada. ¿Dónde va el `assert`, y por qué no en la lista?
2. **El orden que destruye.** En `desapilar`, ¿qué pasa si se escribe `delete cima;` antes de `cima = cima->siguiente;`? Diga qué queda en `cima`.
3. **Dos pilas, un arreglo.** Dos pilas que crecen desde los dos extremos de un mismo arreglo de C casillas. ¿Cuándo se chocan, y cuántos elementos caben entre las dos?
4. **Paréntesis otra vez.** El programa de equilibrio de paréntesis sobre la pila enlazada. ¿Cambia su costo frente a la pila de arreglo? ¿Y su consumo de memoria?
5. **El mínimo en $\Theta(1)$.** Agregue una operación `minimo()` que devuelva el menor elemento de la pila sin recorrerla. Pista: una segunda pila que guarde el mínimo hasta cada punto.

Resumen I

- Una pila se construye sobre el TAD Lista usándola por un solo extremo: `apilar` es `insertar(0, e)`, `desapilar` es `eliminar(0)` y `tope` es `obtener(0)`. No hay que volver a manejar nodos.
- El costo de esas tres no es del TAD: depende de la lista de abajo y del extremo escogido. El arreglo es gratis por el final y cuadrático por el frente; la enlazada, al revés.
- Con la enlazada y el tope al frente, las tres operaciones son $\Theta(1)$ y desaparece el techo de `CAPACIDAD`. El precio es un puntero por elemento.
- La versión directa sobre nodos hace lo mismo sin el salto de llamada, a cambio de escribir el `new`, el `delete` y el destructor.

Resumen II

Lo que sigue

La cola, que atiende por los dos extremos a la vez: por uno entra y por el otro sale. Ningún extremo puede ser el caro, y eso cambia qué lista sirve.

Referencias I


R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 7, pilas: implementación con arreglos y con listas enlazadas.


T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Sección 10.1, pilas y colas.


N. Kalicharan. *Data Structures in C*. 2008. Capítulo 4, pilas y colas.